Sisteme de operare
Laborator 6
System V IPC

1. Creati un nou subdirector *lab6/* in structura de directoare a laboratorului creata anterior (*SO/laborator*) si subdirectoarele aferente *doc src* si *bin*. Nu uitati sa actualizati variabila de mediu PATH pentru a include directorul *SO/laborator/lab6/bin*.

2. Scrieti doua programe C **msg-echod.c** si **msg-echo.c** care implementeaza un serviciu de *echo* folosind cozi de mesaje System V IPC. Programul **msg-echod.c** creeaza o coada de mesaje cu ajutorul *msgget* (parametrul *key* il generati cu functia *ftok*). Apoi apeleaza in bucla functia *msgrcv* pentru a primi mesaje de la clienti pe care le scrie inapoi in coada de mesaje (serviciu de *echo*) folosind *msgsnd*. Programul **msg-echod.c** se lanseaza in background ca serviciu de *echo* dupa cum urmeaza:

```
$ gcc -o echod  msg-echod.c
$ echod keyfile &                    # keyfile e un fisier pe care il creati voi
$ ipcs -q                            # vizualizati coada creata
```

Programul **msg-echo.c** foloseste parametrul primit in linie de comanda (*keyfile*) ca argument pt functia *ftok* si astfel poate identifica si obtine cu ajutorul *msgget* un ID pentru coada de mesaje creata de *echod*. Apoi, citeste in bucla caractere de la tastatura folosind functia *fgets* si asambleaza mesaje cu tipul dat de PID-ul sau (obtinut cu *getpid*) pe care le trimite in coada de mesaje cu ajutorul functiei *msgsnd*. Apoi citeste din coada de mesaje cu ajutorul *msgrcv* ecoul caracterelor trimise catre serviciul *echod* pe care, odata primite, le tipareste pe ecran. Programul se termina cand utilizatorul apasa Ctrl-C.

```
$ gcc -o echo msg-echo.c
$ echo keyfile
```

La final, nu uitati sa faceti curatenie:

```
$ jobs
$ kill %<n>                  # n e numarul de job al echod afisat de comanda
                             # jobs de mai sus
$ ipcs -q
$ ipcrm -q <id>              # id e id-ul cozii de mesaje tiparit de comanda
                             # ipcs de mai sus
```

3. Scrieti un program C **mem-layout.c** care tipareste adresele (aproximative) ale sectiunilor unui program: cod (.text), date neinitializate (.bss), date initializate, stiva, heap (+ program break), memorie partajata. Pentru a realiza acest lucru, tipariti urmatoarele adrese:

- adresa functiei main
- adresa unei variabile globale initializate
- adresele de inceput si sfarsit ale unui array de octeti neinitializat (de dimensiune relativ mare, sa zicem 40000 bytes)
- adresa primei variabile locale declarate in *main* (va fi alocata pe stiva)
- adresa unui vector de 10000 octeti alocat in *main* cu *alloca*
- adresa unui vector de 10000 de octeti alocat dinamic in *main* cu *malloc* + program break-ul (folositi apelul *sbrk*)

- creati o zona de memorie partajata privata (*shmget* cu IPC_PRIVATE) si tipariti valoarea adresei la care se ataseaza in program (obtinuta cu *shmat*).

La final, apelati *shmctl* cu IPC_RMID pentru a sterge zona de memorie partajata din sistem.

Indicatie: adresele de variabile se tiparesc folosind *%p* in formatul *printf*.

4. Rescrieti programul C **race.c** din laboratorul 3 in care doua procese, parinte si copil, scriu concurent propriul string pe ecran, scrierea fiind facuta caracter cu caracter. Pentru a evidentia mai clar race condition-ul, procesele parinte si copil repeta operatia de tiparire pe ecran de un numar de ori, in bucla. Folositi un semafor System V pentru a impiedica producerea race condition-ului si a obtine un output neintretesut pe ecran. La final, apelati *semctl* cu IPC_RMID pentru a sterge semaforul din sistem.

Indicatie: fiind vorba de procese inrudite, puteti crea semaforul cu IPC_PRIVATE.

5. Scrieti un program C **shm.c** care creeaza o zona de memorie partajata intre un proces parinte si un copil. Parintele creeaza zona de memorie partajata (puteti folosi IPC_PRIVATE) si o ataseaza la proces inainte de apelul *fork*. Procesul copil executa un loop cu 10 iteratii in care scrie succesiv, la fiecare iteratie, urmatorul mesaj in zona de memorie partajata:

"this is the child printing " + numarul iteratiei

Pentru a compune asemenea mesaje puteti folosi functia de biblioteca *sprintf*. Mesajele din fiecare iteratie se adauga (*append*) la sfarsitul mesajului scris in iteratia anterioara (cu exceptia primei iteratii).

Procesul parinte executa si el un loop cu 10 iteratii in care, la fiecare iteratie, citeste unul dintre mesajele scrise de procesul copil si le afiseaza pe ecran.

Dezvoltati o solutie folosind semafoare System V care sa permita coordonarea (sincronizarea) accesului celor doua procese la zona de memorie partajata a.i. executia programului sa fie cea anticipata, si anume dupa fiecare mesaj scris de copil in memoria partajata parintele il citeste si il scrie pe ecran. (*Obs*: *ambele procese executa un loop cu 10 iteratii dar nu puteti face nici o presupunere asupra vitezei relative de executie a celor doua procese*).

La final, apelati *shmctl* si *semctl* cu IPC_RMID pentru a sterge memoria partajata si semafoarele din sistem.